Python Task Priority Sorting Function

https://gemini.google.com/app/6a0f78235b8723ec

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Implement sort_tasks_by_priority(tasks). Return a new list of the task dictionaries sorted by priority high, medium, low; within each priority, earlier due dates first and tasks with due_date=None last. Preserve input order for equal keys, do not modify the input list or dictionaries, and return [] for empty input. Assume valid task dictionaries as produced by add_task. Return only the function and any required standard-library imports in one Python code block.

Response:
```
from datetime import date

def sort_tasks_by_priority(tasks):
    priority_order = {"high": 0, "medium": 1, "low": 2}

    def sort_key(task):
        p_val = priority_order.get(task["priority"], 3)
        due = task.get("due_date")
        # due_date is either a date object or None
        # None sorts last within the same priority level
        due_key = (1, date.max) if due is None else (0, due)
        return (p_val, due_key)

    return sorted(tasks, key=sort_key)
```

User prompt: Critique your preceding sort_tasks_by_priority implementation against these requirements: high/medium/low priority order; ascending dates within each tier; None last within its tier, including when a dated task has date.max; stable ties; a new list with no input mutation; empty input. For each criterion give PASS or FAIL with a specific reason or counterexample. Distinguish inspection from tests actually run. Do not invent defects if the code is correct. Then identify any unnecessary complexity under the valid-input assumption. Do not revise the code yet.

Response: Requirement Checklist Critique

Below is an evaluation of the `sort_tasks_by_priority` implementation against each stated criterion based on code inspection (no unit tests were executed):

1. **High / Medium / Low priority order**

- **Verdict:** PASS
- **Reason:** `priority_order = {"high": 0, "medium": 1, "low": 2}` assigns lower numerical ranks to higher priorities. In Python's tuple comparison, lower integers are sorted first (`0 < 1 < 2`), ordering tasks as high, medium, then low.

2. **Ascending dates within each tier**
   - **Verdict:** PASS
   - **Reason:** For tasks with a valid `date` object, `due_key` evaluates to `(0, due)`. Python compares `(0, date_A)` vs `(0, date_B)`, where `date_A < date_B` correctly places earlier dates before later dates.
3. **`None` last within its tier, including when a dated task has `date.max`**
   - **Verdict:** PASS
   - **Reason:** Tasks with `due_date=None` produce `due_key = (1, date.max)`, while tasks with a date object produce `due_key = (0, due)`. Tuple comparison evaluates the first element `0 < 1` before comparing the second element. Even if a task has `due_date = date.max`, its key is `(0, date.max)`, which sorts before `(1, date.max)` because $0 < 1$.
4. **Stable ties (preserve input order for equal keys)**
   - **Verdict:** PASS
   - **Reason:** Python's built-in `sorted()` algorithm (Timsort) is guaranteed to be stable. Tasks with identical `(priority, due_key)` sort keys preserve their original relative position.
5. **Return a new list with no input mutation**
   - **Verdict:** PASS
   - **Reason:** `sorted(tasks, key=sort_key)` constructs and returns a new list of references without invoking `.sort()` or mutating the dictionary elements.
6. **Empty input (`[]`)**
   - **Verdict:** PASS
   - **Reason:** Passing `[]` to `sorted()` returns `[]` immediately without executing `sort_key`.

Unnecessary Complexity Analysis

Under the assumption of valid input (where `due_date` is always either a `date` instance or `None`, and `priority` is always `"high"`, `"medium"`, or `"low"`):

1. **Redundant fallback in `priority_order.get()`**
   - **Detail:** `.get(task["priority"], 3)` includes a default value of `3`. Under the valid-input assumption, direct indexing `priority_order[task["priority"]]` is sufficient.

2. **Redundant second element `date.max` for `None` keys**
    - **Detail:** Using a 2-tuple `(0, due)` vs `(1, date.max)` is slightly redundant. Since the first element (`0` vs `1`) already guarantees that dated tasks sort before `None` tasks, Python never evaluates the second element of `(1, date.max)` when comparing against a dated task `(0, due)`. Thus, `(1, None)` or `(1, 0)` produces identical sorting behavior.

User prompt: Revise the implementation based on that critique. Keep every original requirement, including None sorting after an actual date.max and stable equal keys. Under the stated valid-input assumption, remove unnecessary fallbacks and avoid an unnecessary date import while keeping the code readable. Return only the complete function in one Python code block.

Response:
```
def sort_tasks_by_priority(tasks):
    priority_order = {"high": 0, "medium": 1, "low": 2}

    def sort_key(task):
        due = task["due_date"]
        due_key = (1, 0) if due is None else (0, due)
        return (priority_order[task["priority"]], due_key)

    return sorted(tasks, key=sort_key)
```
